

Final Round Technical Assessment

Overview

This assessment tests your ability to **design, build, collaborate, and reason** about a small but realistic internal system.

We care about:

- Code quality
- Architecture decisions
- UX and usability
- How you work as a pair
- How you handle ambiguity and trade-offs

You will complete:

1. **One paired system task** (groups of 2)
2. **One individual extension task**

Submission: **Git repos + README**

Tech Stack Rules

Paired Task

- Frontend: **React or Angular**
- Backend: **Node.js or C# (.NET)**

Individual Task

- Must use the **other frontend framework**
 - If your pair uses React → your individual task uses Angular
 - If your pair uses Angular → your individual task uses React

Scenario

You are building an **internal operations tool** for a small but growing organisation. The system manages **work requests**, supports **agent workflows**, and must be **extensible** for future growth.

Users (No Auth Required)

Use a simple role switcher:

1. **Admin**
2. **Agent**

Core Entity: Work Request

Each request must include:

- Title
- Description
- Priority (Low / Medium / High / Critical)
- Status (Open / In Progress / Blocked / Done)
- Created date
- Due date
- Assigned agent (optional)

- Tags (e.g., "IT", "Finance", "Urgent", "HR")

Required Features (Paired Task)

Core CRUD & Workflow

- Create, view, update, and delete requests
- View request details
- Assign/unassign agents
- Update status with validation rules (see below)
- Add comments to requests

Search, Filter, Sort

- Search by title, description, and tags
- Filter by:
 - Status
 - Priority
 - Assigned agent
 - Overdue requests
- Sort by:
 - Last updated
 - Priority
 - Due date

Business Rules (New Difficulty Layer)

Implement the following logic:

1. **Status Transitions**
 - Open → In Progress → Done
 - Open/In Progress → Blocked → In Progress → Done
 - You cannot mark a request as Done if it is unassigned.
2. **Overdue Logic**
 - If a request is past its due date and not Done:
 - It should be flagged as **Overdue** in the UI.
 - Admins should see a warning banner on the request.
3. **Priority Escalation**
 - If a request is overdue by more than 3 days:
 - Automatically escalate priority to **Critical** (unless already Critical).
4. **Comment Types**
 - Comments can be:
 - General
 - Status update
 - System-generated (e.g., "Priority escalated automatically")

Data Storage

You may use:

- In-memory store with JSON persistence, **or**
- SQLite

But now:

- You must include **basic migration or versioning logic** (even if simple).
- Explain your data model and trade-offs in the README.

UI/UX Requirements (Harder)

- Role-based UI:
 - Admins can:
 - Assign agents
 - Change any status
 - Delete requests
 - Agents can:
 - Only update requests assigned to them
 - Add comments
 - Mark tasks as Blocked or Done
- Include:
 - Loading states
 - Empty states
 - Error states
 - Confirmation modals for destructive actions
- Include a **dashboard view**:
 - Summary cards (e.g., total open, overdue, blocked, done)
 - A small visual indicator (bar, badge, or chart — keep simple, no chart library required)

Testing (Stronger Expectation)

You must include:

- At least:
 - 3 meaningful backend tests
 - 2 meaningful frontend tests
- Tests should cover:
 - One business rule
 - One failure scenario
 - One happy path

Collaboration Requirements (Paired Task)

In your README, include:

- How you split the work
- How you communicated
- What trade-offs you made
- What you would improve with more time

Bonus (not required but impressive):

- Use GitHub Issues, Projects, or a task board and include screenshots.

Individual Task (Creative Extension)

Each candidate must build **one independent feature module** using the other frontend framework.

Choose **one**:

Option A — Activity Log Viewer

Build a UI that:

- Displays a timeline of request activity:
 - Status changes
 - Assignments

- Priority escalations
 - Comments
- Includes:
 - Filters (by event type, user, date)
 - Search
 - Clean, readable timeline UI

Option B — Smart Request Triage

Build a UI that:

- Suggests:
 - Priority
 - Due date
 - Suggested agent
- Based on:
 - Tags
 - Keywords in description
 - Simple rules or heuristics (no ML required)
- Include:
 - Explanation of how the suggestion was made
 - Ability to override suggestions

Deliverables

Each pair must submit:

- Repository with:
 - Frontend
 - Backend
- README including:
 - Setup instructions
 - Architecture overview
 - Data model
 - Business rules implemented
 - Known limitations
 - Collaboration breakdown
- Optional:
 - Screenshots
 - Short demo video
 - Link to live deployment (if applicable)

Each individual must submit:

- Separate repo for their extension task
- README explaining:
 - Design choices
 - Trade-offs
 - What they'd improve

Enhanced Individual Challenge: Public API Integration (Final Round)

Goal

Build a **production-quality data exploration tool** using a public API. This task tests:

- Real-world API consumption
- UI/UX design
- State management
- Error handling
- Data transformation and insight generation
- Code structure and scalability

Tech Rules

- Must use the **frontend framework not used in the group task**.
- No backend required — frontend-only.
- You may use a UI library, but justify your choice.

API Options (Choose ONE)

Option 1 — GitHub API (Serious, High Signal)

Build a **Developer Intelligence Dashboard**.

Core Features

- Search public repositories with:
 - Language filter
 - Stars range filter
 - Sorting (stars, forks, updated date)
- Paginated list view
- Repository detail view:
 - Stars, forks, open issues
 - README preview
 - Languages breakdown

Creative / Harder Layer

- Add a **Repository Health Score** (0–100) based on:

- Issue count vs closed issues
 - Commit recency
 - Stars vs forks ratio
- Display the score with a clear explanation of how it's calculated.
- Add a **Compare View**:
 - Compare two repositories side by side.

Option 2 — PokéAPI (Fun, Design-Heavy)

Build a **Pokémon Battle Intelligence Tool**.

Core Features

- Browse/search Pokémon
- List + detail views
- Show stats, abilities, types

Creative / Harder Layer

- Add a **Battle Readiness Score** based on:
 - Total stats
 - Type advantages (simple rules)
 - Speed vs defense weighting
- Allow users to:
 - Compare two Pokémon
 - See a predicted winner with explanation
- Add a **Type Explorer View**:
 - Group Pokémon by type
 - Show strongest Pokémon per type

Option 3 — NASA / Space APIs (Educational, Data-Messy)

Build a **Space Data Explorer Dashboard**.

Core Features

- Browse/search items (images, missions, objects)
- List + detail views
- Handle missing or inconsistent data gracefully

Creative / Harder Layer

- Add a **Discovery Timeline**:
 - Visual timeline of missions or discoveries
- Add a **Classification View**:
 - Group by mission type, year, or agency
- Add a **Data Confidence Indicator**:
 - Show how complete or reliable a data entry is.

New Difficulty Layer (Applies to ALL Options)

Required Advanced Features

1. **State Management**
 - Use proper global state (Redux, Zustand, Signals, or equivalent).
 - Explain why you chose it.
2. **Caching & Performance**
 - Implement basic client-side caching.
 - Avoid refetching the same data unnecessarily.
3. **Resilience**
 - Handle:
 - Network failures
 - API rate limits
 - Partial/malformed responses
 - Show user-friendly recovery paths (retry, fallback UI).
4. **Accessibility**
 - Keyboard navigation
 - ARIA labels where relevant
 - Contrast-friendly UI
5. **Testing**
 - At least:
 - 2 unit tests
 - 1 integration or UI test
 - Must test:
 - One failure case
 - One transformation/computation

UI/UX Expectations

- Clear layout hierarchy
- Loading skeletons
- Empty states
- Error states with recovery actions
- Responsive design (mobile + desktop)
- At least one **data visualization** (simple chart, badge system, or progress bar — no heavy chart libraries required)

Deliverables

- Git repo
- README explaining:
 - API choice
 - Architecture decisions
 - Data transformations
 - Caching strategy
 - Known limitations
 - What you would add with more time

Optional:

- Screenshots
- Short demo video
- Live deployment link

Frontend Deliverable (Applies to BOTH group and individual tasks)

Add this requirement:

Design Principles Deliverable

Each frontend must include a short section in the README titled:

“Design Principles Applied”, covering the following five principles:

1. **Clarity** – How the UI avoids confusion and reduces cognitive load.
2. **Consistency** – How patterns, components, and behaviors are reused.
3. **Feedback** – How the system communicates state (loading, success, errors).
4. **Accessibility** – How the UI supports diverse users (keyboard, contrast, screen readers).
5. **Efficiency** – How the UI helps users complete tasks quickly and with minimal friction.

For each principle, candidates must:

- Explain how it was applied.
- Reference at least one concrete UI example or screen.